The ASOF Join

A Study Guide for Implementing Temporal Nearest-Match Joins in PostgreSQL

CS631 Project — Deepjyoti Kakoti (26M0774)
Debasish Bordoloi (26M0776) Shubh Karman Singh (26M0843)

October 26, 2026

Abstract

An ASOF join matches each row of a left relation to the *nearest preceding* row of a right relation, ordered by time and optionally partitioned by a key. It is the natural operator for aligning two unsynchronised time series — trades against quotes, readings against calibration events, facts against slowly-changing dimensions. DuckDB, ClickHouse, Snowflake, kdb+ and QuestDB all provide it; PostgreSQL does not, and users fall back on a hand-written `LEFT JOIN LATERAL ... ORDER BY ... LIMIT 1`.

This document is the study companion for a two-phase project that adds ASOF join to PostgreSQL 17. It develops the operator from its formal definition, explains why the existing workaround is both awkward and asymptotically worse than it needs to be, walks through the stages of PostgreSQL's query processor, and then treats each implementation phase as a technique worth understanding in its own right: Phase 1 as a *semantics-preserving source-to-source rewrite*, Phase 2 as a *sort-merge streaming algorithm* with a proof of correctness and a cost model. The emphasis throughout is on *why* each technique works, not merely what to type.

Contents

1 The Problem **3**
- 1.1 A motivating scenario . . . 3
- 1.2 Formal definition . . . 3
- 1.3 Why ordinary joins cannot express this . . . 4
- 1.4 Exercises . . . 5

2 How Other Systems Do It **6**
- 2.1 Why we follow Snowflake . . . 6
- 2.2 Semantic choices to pin down . . . 7

3 What PostgreSQL Does Today **8**
- 3.1 The LATERAL workaround . . . 8
- 3.2 Cost analysis . . . 8
- 3.3 A third problem: planning . . . 9
- 3.4 Exercises . . . 9

4 Anatomy of a PostgreSQL Query **10**
- 4.1 Stage by stage . . . 10
- 4.2 Two structures worth studying before Phase 2 . . . 11
- 4.3 Exercises . . . 11

5 Phase 1 — The Rewrite Technique **12**
- 5.1 Why rewrite at all? . . . 12
- 5.2 Grammar mechanics . . . 12
  - 5.2.1 Keyword categories . . . 12
  - 5.2.2 The production . . . 13
  - 5.2.3 Conflicts . . . 13
- 5.3 Parse-tree surgery . . . 13
  - 5.3.1 The translation . . . 13
  - 5.3.2 Direction, strictness, and the four operators . . . 14
  - 5.3.3 Why the equality keys go *inside* the subquery . . . 14
- 5.4 Correctness of the translation . . . 15
- 5.5 Validation and error messages . . . 15
- 5.6 Exercises . . . 15

6 Phase 2 — The Merge Technique **17**
- 6.1 The insight . . . 17
- 6.2 The algorithm . . . 17
  - 6.2.1 Why no mark/restore . . . 18
- 6.3 Correctness . . . 18
- 6.4 Complexity . . . 19

6.5 A worked example . . . 20
6.6 Exercises . . . 20

7 Making the Planner Choose It **21**
7.1 Pathkeys: the planner's model of order . . . 21
7.2 Path generation . . . 21
7.3 Why ASOF constrains join order . . . 22
7.4 The cost model . . . 22
7.5 The GUC . . . 23
7.6 Exercises . . . 23

8 Correctness Corners **24**
8.1 Exercises . . . 25

9 Measuring It **26**
9.1 Reading the plans . . . 26
9.2 Benchmark design . . . 26
9.3 Differential testing . . . 27
9.4 Exercises . . . 27

A Source File Map **28**

B Glossary **29**

Chapter 1

The Problem

1.1 A motivating scenario

Consider a market-data warehouse with two tables. Trades record executions; quotes record the best bid and ask prevailing at an instant. Both are append-only streams, both are timestamped, and — crucially — their timestamps do not line up.

```
CREATE TABLE trades (symbol text, ts timestamptz, price numeric, size int);
CREATE TABLE quotes (symbol text, ts timestamptz, bid numeric, ask numeric);
```

The analytical question is almost always the same: *for each trade, what was the market showing at the moment it executed?* That requires, per trade, the most recent quote for the same symbol at or before the trade's timestamp.

trades			**quotes**			
symbol	ts	price	symbol	ts	bid	ask
AAPL	09:30:02	189.10	AAPL	09:30:00	189.00	189.05
AAPL	09:30:07	189.20	AAPL	09:30:05	189.15	189.20
MSFT	09:30:04	410.50	AAPL	09:30:09	189.25	189.30
			MSFT	09:30:01	410.40	410.45

Table 1.1: A trade at 09:30:07 must match the quote at 09:30:05 — not 09:30:09, which had not happened yet, and not 09:30:00, which was stale.

The same shape recurs far outside finance:

- **IoT / telemetry.** Align a dense sensor stream with a sparse stream of calibration or configuration changes: which calibration was in force when this reading was taken?
- **Slowly-changing dimensions.** A fact row must join the dimension version that was current at the fact's event time, not the version current today.
- **Experiment analysis.** Attribute each event to the most recent assignment, exposure or consent record preceding it.
- **Log correlation.** Pair an error with the last deploy before it.

In every case one side is "events" and the other is "state that changes over time", and the join condition is *temporal precedence plus recency*, not equality.

1.2 Formal definition

Fix two relations: L (the *probe* or outer side) and R (the *build* or inner side). Let t_L and t_R be the match attributes — the timestamps — and let k_L, k_R be (possibly empty) tuples of partition-key attributes.

Definition 1.1 (ASOF match)**.** For a left tuple $\ell \in L$, define the *candidate set*

$$C(\ell) \;=\; \{\, r \in R \;:\; k_R(r) = k_L(\ell) \;\wedge\; t_R(r) \le t_L(\ell) \,\}.$$

The ASOF match of ℓ is

$$m(\ell) \;=\; \begin{cases} \arg\max_{r \in C(\ell)} t_R(r), & C(\ell) \neq \varnothing, \\ \bot, & C(\ell) = \varnothing, \end{cases}$$

where $\bot$ denotes "no match". The ASOF *left join* is

$$L \bowtie^{\text{left}}_{\text{asof}} R \;=\; \{\, \ell \circ m(\ell) \;:\; \ell \in L \,\},$$

with $m(\ell)$ replaced by a tuple of NULLs when $m(\ell) = \bot$; the *inner* variant drops those rows instead.

Three properties of Definition 1.1 drive everything that follows.

(P1) **Exactly one match per left row.** The output cardinality of the left variant is precisely $|L|$, independent of $|R|$. This is unusual: it makes ASOF a *function* from L to $R \cup \{\bot\}$ rather than a relation, and it means the planner can estimate output rows with certainty.

(P2) **Asymmetry.** arg max picks the *latest preceding* right row. Swapping L and R gives a different answer, so ASOF is *not commutative* — a fact that constrains join reordering (Chapter 7).

(P3) **Order dependence.** The operator is defined in terms of $\le$ and max on t, so it only makes sense for a type with a total order — in PostgreSQL terms, a type with a `btree` operator family. Hash equality is not enough.

Remark 1.2 (The four directions)**.** Definition 1.1 uses $\le$ ("the latest quote at or before the trade"), which is the common case. Three variants follow by symmetry: $<$ (strictly before), $\ge$ and $>$ (the *earliest following* row, i.e. arg min). Our syntax admits all four: `MATCH_CONDITION (l.ts >= r.ts)` is $\le$ read from the left row's point of view. Everything in this document is stated for $\le$; the others are obtained by reversing the comparison and the sort direction, and by excluding the equality case for the strict forms.

1.3 Why ordinary joins cannot express this

Equi-join is too strict

```
SELECT * FROM trades t JOIN quotes q
  ON t.symbol = q.symbol AND t.ts = q.ts;    -- almost always empty
```

Two independent streams rarely share exact timestamps. With microsecond resolution the match probability is essentially zero, and the result is an empty relation — a silent wrong answer rather than an error.

Range join is too permissive

Relaxing equality to inequality finds *every* preceding quote, not the nearest one:

```
SELECT * FROM trades t JOIN quotes q
  ON t.symbol = q.symbol AND q.ts <= t.ts;    -- many-to-many blow-up
```

The damage is quantitative, not just semantic. Suppose n trades and m quotes are spread uniformly over the same interval, within a single symbol. A trade at quantile $u \in [0,1]$ of the interval matches roughly um quotes, so the expected intermediate cardinality is

$$\sum_{i=1}^{n} \mathbb{E}[u_i m] \;=\; n\,m \int_0^1 u\,du \;=\; \frac{nm}{2}.$$

For $n = m = 10^6$ that is 5×10^{11} intermediate rows, to be reduced back to 10^6. The work is quadratic in the data size although the answer is linear in it.

Filtering the range join with a window function

The standard SQL repair picks the nearest row out of the blow-up:

```
SELECT * FROM (
  SELECT t.*, q.bid, q.ask,
         row_number() OVER (PARTITION BY t.symbol, t.ts ORDER BY q.ts DESC) AS rn
  FROM trades t JOIN quotes q
    ON t.symbol = q.symbol AND q.ts <= t.ts
) s WHERE rn = 1;
```

This is *correct*, and it is how the operator is often emulated in systems without it. But it materialises the $nm/2$ intermediate rows before discarding all but n of them: the filter is applied after the damage. No amount of planner cleverness recovers the linear algorithm from this formulation, because the window function is opaque to the join planner.

Key idea. ASOF is not sugar. The three SQL encodings above are respectively wrong, quadratic, and quadratic-then-filtered. A dedicated operator is what allows the engine to exploit (P1)–(P3) and compute the answer in one ordered pass.

1.4 Exercises

1. Show that the ASOF left join of Definition 1.1 always has exactly $|L|$ rows, even when R is empty.

2. Give a pair of relations for which $L \bowtie_{\text{asof}} R \neq R \bowtie_{\text{asof}} L$ even after renaming columns, confirming (P2).

3. The window-function formulation above partitions by `(t.symbol, t.ts)`. What goes wrong if two distinct trades share a symbol and timestamp, and how would you fix it?

4. Suppose t_R values are unique within a key. Does the arg max in Definition 1.1 still need tie-breaking? What if they are not unique?

Chapter 2

How Other Systems Do It

Borrowing a design is cheaper than inventing one, and divergence from established semantics is a bug report waiting to happen. Table 2.1 summarises the landscape.

System	Syntax	Keys	Notes
DuckDB	`ASOF JOIN r ON l.k = r.k AND l.ts >= r.ts`	equalities in `ON`	Inequality lives in `ON`; supports `LEFT`; one inequality required.
ClickHouse	`ASOF LEFT JOIN r ON l.k = r.k AND l.ts >= r.ts`	equalities in `ON`	Inequality must be last; requires at least one equality key.
Snowflake	`ASOF JOIN r MATCH_CONDITION (l.ts >= r.ts) ON l.k = r.k`	separate `ON`	Match condition is syntactically distinct from the keys.
kdb+/q	`aj[`k`ts; l; r]`	positional	Function, not SQL; requires sorted, attribute-tagged tables.
QuestDB	`l ASOF JOIN r ON (k)`	`ON (k)`	Designated timestamp column is implicit in the table definition.

Table 2.1: ASOF join across systems.

2.1 Why we follow Snowflake

The project adopts Snowflake's shape:

```
FROM trades t ASOF LEFT JOIN quotes q
  MATCH_CONDITION (t.ts >= q.ts)
  ON t.symbol = q.symbol
```

Three reasons, all of them implementation-driven:

1. **Separation of concerns.** The DuckDB/ClickHouse style mixes one inequality and n equalities in a single `ON` clause, then requires the analyser to pick the inequality apart from the rest and reject anything else. Keeping `MATCH_CONDITION` separate turns a validation problem into a grammar problem, and grammar problems produce better error messages and cleaner positions (`parser_errposition`).

2. **No overloading of `ON`.** In PostgreSQL, `ON` already feeds `transformJoinOnClause` and ultimately the planner's restriction-clause machinery, where an inequality would be treated as an ordinary filter. Giving the match condition its own clause means it can be stored in its own `JoinExpr` field and never mistaken for a qual.

3. **Room for extension.** A distinct clause gives an obvious home for future `TOLERANCE`/window bounds ("match only within 5 seconds"), which several systems offer and which we may add later.

The cost is that queries are not copy-paste portable from DuckDB. We judge discoverable errors and a clean parse tree worth more than cross-dialect portability in a teaching implementation.

2.2 Semantic choices to pin down

Systems quietly disagree on the corners. We must choose, document, and test each:

Corner	Our choice
NULL in match column	Never matches (NULL comparisons are unknown, so the row is filtered).
NULL in a partition key	Never matches, consistent with = semantics.
Ties on t_R within a key	Implementation-defined: one arbitrary tuple. Documented, and Phase 2 must agree with Phase 1.
Empty right side	Left variant returns all left rows NULL-extended.
Strict operators ($<$, $>$)	Equal timestamps excluded.
Zero equality keys	Allowed: a single global ordered stream.

Table 2.2: Semantic decisions, to be enforced by the regression suite.

Remark 2.1 (On ties)**.** Picking "an arbitrary tuple" is defensible but must be *consistently* arbitrary across phases, or the differential test of Chapter 9 will flag diffs that are not bugs. The practical rule: Phase 1's `ORDER BY ts DESC LIMIT 1` and Phase 2's "last candidate seen in ascending order" happen to agree when the sort is stable on the same key set, and the tests compare multisets after excluding tie rows.

Chapter 3

What PostgreSQL Does Today

3.1 The LATERAL workaround

Without ASOF, the idiomatic PostgreSQL formulation is a correlated subquery in the `FROM` clause:

```
SELECT t.symbol, t.ts, t.price, q.bid, q.ask
FROM trades t
LEFT JOIN LATERAL (
    SELECT q.bid, q.ask
    FROM quotes q
    WHERE q.symbol = t.symbol
      AND q.ts    <= t.ts
    ORDER BY q.ts DESC
    LIMIT 1
) q ON true;
```

`LATERAL` is what makes this legal: it permits the subquery to reference `t`, the relation to its left. Semantically the subquery is re-evaluated for each row of `trades`, and `ORDER BY ... LIMIT 1` performs the arg max of Definition 1.1 directly. `LEFT JOIN ... ON true` supplies the NULL-extension when the subquery returns no row.

This is correct. It is also:

- **verbose** — eleven lines encoding one idea, with the correlation, the sort direction, the `LIMIT` and the `ON true` each independently easy to get wrong;
- **fragile** — writing `ORDER BY q.ts` instead of `q.ts DESC` yields the *oldest* quote, silently;
- **dependent on an index** for acceptable performance, as the next section shows.

3.2 Cost analysis

The plan is a nested loop with the subquery on the inner side.

With a suitable index. Given `CREATE INDEX ON quotes (symbol, ts DESC)`, each inner evaluation is a `Limit` over an `Index Scan` whose scan key is `symbol = $1 AND ts <= $2`. Because the index is ordered, the scan descends to the right leaf and stops after one tuple. Per outer row the cost is one index descent, $O(\log m)$, giving

$$T_{\text{lateral}} \;=\; O(n \log m)\,.$$

Each descent is a handful of random page accesses; with $n = 10^6$ that is 10^6 independent descents, and the constant is dominated by cache misses on the upper index levels and by per-tuple executor overhead for the `Limit`/`Index Scan` node pair.

Without one. The inner side becomes a `Sort` over a filtered `Seq Scan` (or at best a `Top-N heapsort`), re-executed per outer row:

$$T_{\text{lateral}}^{\text{no index}} \;=\; O(n \cdot m)\,,$$

which for $n = m = 10^6$ is 10^{12} tuple comparisons — hours, not seconds. Nested-loop rescans are not amortised; PostgreSQL will reuse the inner `Materialize` node only when the inner side is *not* correlated, and here it necessarily is.

Key idea. The workaround's shape forces a *per-row search*. Even in the good case it pays $\log m$ random descents n times, because nothing tells the engine that consecutive outer rows probe nearby positions in the same ordered inner relation. Phase 2's whole contribution is to exploit exactly that locality.

3.3 A third problem: planning

Beyond asymptotics, the LATERAL form limits the planner. The subquery is a black box with a `LIMIT`: it cannot be hash-joined, cannot be parallelised across the inner side, and its selectivity is estimated through the generic subquery path. The optimiser cannot, for instance, decide to sort both inputs once — that plan is not in the search space, because the query as written does not describe it.

3.4 Exercises

1. Derive the $nm/2$ figure of Chapter 1 for the range-join formulation, then compare it with the $n \log m$ of the LATERAL form at $n = m = 10^6$. By what factor do they differ?
2. Why can PostgreSQL not place a `Materialize` node above the correlated inner side of this nested loop?
3. Rewrite the LATERAL query for the "earliest following quote" variant. Which two things change?

Chapter 4

Anatomy of a PostgreSQL Query

To extend the engine one must know where to cut. PostgreSQL processes a query in five stages, each consuming one data structure and producing the next.

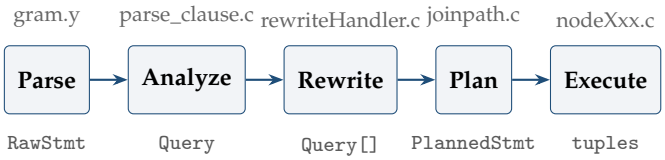


Figure 4.1: The five stages, the structure each produces, and a representative source file.

4.1 Stage by stage

1. Parse (`src/backend/parser/gram.y`, `scan.l`). A Bison grammar turns text into a *raw parse tree*: a tree of nodes that mirrors the syntax and resolves *nothing*. Table names are strings, column references are `ColumnRef` lists, and no catalog is consulted. The parser's only job is to decide whether the text is well-formed and to record its shape. A `FROM` clause join becomes a `JoinExpr` with `larg`, `rarg`, `jointype`, `quals` and `usingClause`.

2. Analyze (`parser/analyze.c`, `parse_clause.c`, `parse_expr.c`). Semantic analysis resolves names against the catalog, assigns types, and produces a `Query`. Range-table entries (`RangeTblEntry`) replace table names; `Var` nodes with `(varno, varattno)` replace column references; operators are resolved to specific `Oid`s through overload resolution. The `FROM` clause is processed by `transformFromClauseItem()`, which is recursive over `JoinExpr` — this is the function Phase 1 hooks.

3. Rewrite (`rewrite/rewriteHandler.c`). Views are expanded and rules applied, `Query` in, `Query` out. The ASOF project does not touch this stage, but it matters for one reason: a view defined over an ASOF join must survive expansion, which is why the node must deparse correctly (`ruleutils.c`).

4. Plan (`optimizer/`). The planner converts a `Query` into a `PlannedStmt`. Internally it works with `RelOptInfo` (a relation, base or join) and `Path` (one way of producing that relation, with a cost and a sort order). `add_paths_to_joinrel()` in `path/joinpath.c` proposes every join strategy for a pair of relations; the cheapest surviving `Path` is turned into a `Plan` by `plan/createplan.c`. Phase 2 adds a `Path` type here and teaches `path/costsize.c` what it costs.

5. Execute (`executor/`). The `Plan` tree is instantiated as a `PlanState` tree and pulled one tuple at a time: each node's `ExecProcNode` returns a `TupleTableSlot` or NULL at end of stream. This is the Volcano iterator model, and it is why a streaming merge algorithm fits so naturally. Phase 2 adds `executor/nodeAsofjoin.c`.

Stage	Structure	Phase 1 touches	Phase 2 touches
Parse	`RawStmt`, `JoinExpr`	grammar, new fields	—
Analyze	`Query`, `Var`, `RTE`	validation + rewrite	emit `JOIN_ASOF`
Rewrite	`Query`	deparse only	deparse only
Plan	`Path`, `Plan`, `PathKey`	—	path, cost, plan
Execute	`PlanState`, slots	—	new node

Table 4.1: Where each phase intervenes. Phase 1 deliberately stops at stage 2.

Key idea. Phase 1 and Phase 2 differ in *which stage they stop at*. Phase 1 rewrites the parse tree and lets stages 3–5 run untouched; Phase 2 carries a new concept all the way to stage 5. That is why Phase 1 is low-risk and Phase 2 is where the performance is.

4.2 Two structures worth studying before Phase 2

`PathKey`. A `PathKey` is the planner's representation of "this path produces tuples ordered by expression e, using operator family f, ascending or descending, nulls first or last". A list of them is a sort order. Crucially, `PathKeys` are *canonicalised*: two paths claiming the same order share pointer-equal `PathKey` objects, so order compatibility is tested by list-prefix comparison rather than expression matching. This is the mechanism by which a merge join avoids a sort when an index already supplies the order — and the mechanism Phase 2 reuses.

`TupleTableSlot`. A slot is a holder for one tuple, possibly virtual (array of `Datums`), possibly a pointer into a buffer page. Slots are *overwritten* by the next `ExecProcNode` call on the same child. Any tuple a node needs to remember across calls must be copied with `ExecCopySlot` into a slot the node owns — a rule that Phase 2's candidate buffer depends on absolutely (§6.2).

4.3 Exercises

1. Which stage would reject `SELECT * FROM nonexistent`? Which would reject `SELECT FROM WHERE`?

2. Why must node types be registered for `copyObject` and `outNode` even if the plan is never written to disk? (Hint: prepared statements, parallel workers.)

3. Explain, in terms of Figure 4.1, why Phase 1 cannot improve performance over the hand-written LATERAL query.

Chapter 5

Phase 1 — The Rewrite Technique

5.1 Why rewrite at all?

It is tempting to go straight to the native operator. Rewriting first buys four things:

1. **Zero executor risk.** The rewrite produces a plan the engine already knows how to run correctly. No new crash surface, no new memory-management bugs.
2. **A reference implementation.** Phase 2 must agree with *something*. The rewrite is an executable specification of Definition 1.1, and Chapter 9's differential test compares the two implementations directly — an oracle we would otherwise lack.
3. **Early feedback on syntax.** The grammar, the error messages and the documentation are all exercised before any planner work begins. If the syntax is wrong, we learn it cheaply.
4. **A fallback.** Behind a GUC, the rewrite remains available when the native path is disabled or inapplicable.

This is a standard compiler technique — *desugaring*, or source-to-source translation into a core language. The new construct is defined by the translation, and the existing machinery does the work.

5.2 Grammar mechanics

5.2.1 Keyword categories

PostgreSQL classifies every keyword into one of four categories in `src/include/parser/kwlist.h`:

Category	Usable as identifier?	Example
`UNRESERVED_KEYWORD`	yes	`ASOF` (ours)
`COL_NAME_KEYWORD`	as a column name only	`BETWEEN`
`TYPE_FUNC_NAME_KEYWORD`	not as type/function name	`LEFT`
`RESERVED_KEYWORD`	no	`SELECT`

Table 5.1: Keyword categories, from least to most intrusive.

We register `ASOF` and `MATCH_CONDITION` as **unreserved** and add them to the `unreserved_keyword:` production in `gram.y`. The reason is compatibility: an existing application may well have a column called `asof` or `match_condition`, and reserving the word would break it. Unreserved keywords are distinguishable by the parser from context alone.

Key idea. Adding a keyword is a compatibility decision, not a syntax decision. Always start at unreserved and escalate only if the grammar genuinely cannot be made unambiguous — and treat "cannot" as a claim to be proved by Bison's conflict report, not assumed.

5.2.2 The production

The join syntax lives in the `joined_table` production. The addition is, in essence:

```
joined_table:
    ...
  | table_ref ASOF opt_outer_join_type JOIN table_ref
       MATCH_CONDITION '(' a_expr ')' opt_asof_on
     {
         JoinExpr *n = makeNode(JoinExpr);
         n->jointype       = $3;         /* JOIN_INNER or JOIN_LEFT */
         n->isAsof         = true;
         n->larg           = $1;
         n->rarg           = $5;
         n->matchCondition = $8;
         n->quals          = $10;        /* may be NULL */
         $$ = (Node *) n;
     }
```

Two design points deserve comment.

`MATCH_CONDITION` **is mandatory in the grammar.** Making it optional and checking later would produce a worse error ("ASOF join requires a match condition", with no position) than letting Bison report a syntax error at the exact token. Push validation into the grammar wherever the grammar can express it.

`RIGHT`/`FULL` **are accepted by the grammar and rejected by the analyser.** This looks backwards, but it produces the better message. If the grammar excluded them, `ASOF RIGHT JOIN` would yield "syntax error at or near RIGHT", which tells the user nothing. Accepting it and then raising ASOF *join does not support RIGHT outer joins* with a hint is strictly more helpful. The rule of thumb inverts the previous one: push validation into the analyser when the analyser can say something the grammar cannot.

5.2.3 Conflicts

`gram.y` is an LALR(1) grammar with a known, non-zero baseline conflict count (declared via `%expect`). Any *new* shift/reduce or reduce/reduce conflict means the grammar has become ambiguous at one token of lookahead, and must be fixed rather than suppressed. Because `ASOF` appears immediately after a complete `table_ref` — the same position as `LEFT`, `INNER`, `CROSS` — and is followed by a fixed token sequence, this addition is conflict-free. Verify, do not assume: build `src/backend/parser` and compare Bison's report against the baseline.

5.3 Parse-tree surgery

5.3.1 The translation

The heart of Phase 1 is a structural rewrite performed in a new `src/backend/parser/parse_asof.c`, invoked from `transformFromClauseItem()` *before* the right-hand side is transformed. Given

```
L ASOF LEFT JOIN R MATCH_CONDITION (L.ts >= R.ts) ON L.k = R.k
```

we construct the parse tree that the following text would have produced:

```
L LEFT JOIN LATERAL (
    SELECT * FROM R
    WHERE R.ts <= L.ts AND R.k = L.k
    ORDER BY R.ts DESC
    LIMIT 1
) ON true
```

Concretely: a `RangeSubselect` wrapping a `SelectStmt` whose `fromClause` is the original `rarg`, with `lateral = true`, substituted as the `rarg` of an ordinary `JoinExpr` with `quals = makeBoolConst(true, false)`.

Key idea. Rewriting the *raw* parse tree, not the analysed `Query`, is what makes this cheap. Name resolution, type resolution, `LATERAL` validity checking, lateral reference marking and subquery planning are all performed afterwards by existing code that has no idea `ASOF` exists.

5.3.2 Direction, strictness, and the four operators

The match condition is written from the left row's point of view; the subquery's `WHERE` needs it from the right row's. Table 5.2 is the whole of it.

`MATCH_CONDITION`	**Meaning**	**Subquery** `WHERE`	`ORDER BY`
`l.ts >= r.ts`	latest at or before	`r.ts <= l.ts`	`r.ts DESC`
`l.ts > r.ts`	latest strictly before	`r.ts < l.ts`	`r.ts DESC`
`l.ts <= r.ts`	earliest at or after	`r.ts >= l.ts`	`r.ts ASC`
`l.ts < r.ts`	earliest strictly after	`r.ts > l.ts`	`r.ts ASC`

Table 5.2: The four admissible match conditions and their translations.

5.3.3 Why the equality keys go *inside* the subquery

This is the subtlest point in Phase 1, and a natural place to get it wrong. One might place `L.k = R.k` in the outer join's `ON` clause instead:

```
L LEFT JOIN LATERAL (
    SELECT * FROM R WHERE R.ts <= L.ts ORDER BY R.ts DESC LIMIT 1
) q ON L.k = q.k            -- WRONG
```

The result is wrong, not merely slower. The subquery now computes "the latest preceding row of R *over all keys*", and the outer `ON` then discards it if its key happens to differ — producing NULL where a correct match exists. The arg max of Definition 1.1 is taken over the candidate set $C(\ell)$, which is *already restricted by the key*; restricting afterwards does not commute with arg max.

Lemma 5.1 (Key placement). *Let σ_θ be selection and top_1^t the "highest t" operator. Then $\mathrm{top}_1^t(\sigma_\theta(R)) \neq \sigma_\theta(\mathrm{top}_1^t(R))$ in general: the left side returns the best θ-satisfying row, the right side returns the globally best row if it satisfies θ and nothing otherwise.*

This lemma is why the predicate must be pushed *through* the `LIMIT`, i.e. into the subquery, where it filters before the arg max. It also explains why PostgreSQL's own optimiser will never perform this pushdown for us: predicate pushdown past a `LIMIT` is unsound, and `allpaths.c` correctly refuses it.

5.4 Correctness of the translation

Theorem 5.2 (Rewrite equivalence)**.** *For a match condition $t_L \geq t_R$ and key equality $k_L = k_R$, the LATERAL translation of §5.3.1 produces exactly the relation of Definition 1.1, up to the choice among tied t_R values.*

Proof sketch. Fix $\ell \in L$. By the semantics of `LATERAL`, the subquery is evaluated with ℓ's columns bound as constants. Its `WHERE` clause then selects exactly $\{\, r \in R : k_R(r) = k_L(\ell) \wedge t_R(r) \leq t_L(\ell) \,\} = C(\ell)$, which is Definition 1.1's candidate set. `ORDER BY t DESC LIMIT 1` returns a tuple of $C(\ell)$ with maximal t_R when $C(\ell) \neq \varnothing$ — i.e. an element of arg max — and the empty relation otherwise. `LEFT JOIN ... ON true` pairs ℓ with that tuple, or NULL-extends it when the subquery is empty, matching $m(\ell) = \bot$. Since the construction is per-ℓ and the left join preserves every ℓ exactly once, the two relations agree tuple-for-tuple. Where several r share the maximal t_R, both formulations select one arbitrarily; agreement holds up to that choice. □

Remark 5.3. The restriction to *one* top-level comparison in the match condition (enforced by the analyser) is what makes the proof go through: a conjunction or disjunction of inequalities would not reduce to a single `ORDER BY` direction. This is also exactly the restriction Phase 2 needs, which is why it is checked in Phase 1 rather than deferred.

5.5 Validation and error messages

The analyser must verify, with a `parser_errposition()` on the offending node:

1. the match condition is a single `OpExpr` whose operator is one of `>=`, `>`, `<=`, `<`;
2. its left operand references only L and its right operand only R — checked with `pull_varnos` against the respective namespace items — or vice versa, in which case the comparison is flipped;
3. the operand types share a `btree` operator family, so the comparison has a total order. This matters for Phase 1 only insofar as `ORDER BY` needs it, but it is *essential* for Phase 2's merge, so it is rejected early rather than becoming a planner-time failure;
4. the `ON` clause, if present, is a conjunction of plain L-to-R equalities;
5. the join type is `INNER` or `LEFT`.

Every one of these gets a regression test with stable expected output. An error message is an interface: once released it is as hard to change as the syntax itself.

5.6 Exercises

1. Why does the translation use `SELECT *` over the original `rarg` rather than enumerating columns? What would break with an enumerated list?

2. Construct a two-row, two-key counterexample demonstrating the key-placement lemma concretely.

3. The analyser flips the comparison when the operands appear in the opposite order. Write out the four flipped forms and confirm they map onto Table 5.2.

4. Phase 1 performs its rewrite before transforming `rarg`. What goes wrong if it runs after?

Chapter 6

Phase 2 — The Merge Technique

6.1 The insight

Key idea. If both inputs are sorted by (k, t) ascending, then as the outer cursor advances, the ASOF match also advances monotonically. The nearest preceding inner row is simply *the last inner row seen before the boundary* — so a single synchronised pass over both inputs suffices, with no search and no backtracking.

The LATERAL plan re-searches the inner relation from scratch for every outer row, throwing away everything it learned from the previous one. But the outer rows arrive in increasing t order, so each successive probe lands at or after the previous one. Maintaining a single inner cursor plus one remembered tuple replaces n independent $O(\log m)$ descents with $n + m$ sequential steps.

6.2 The algorithm

```
AsofJoin(outer, inner):
  cand       <- EMPTY          # best candidate for the current outer row
  cand_key   <- NULL           # the key group cand belongs to
  r          <- next(inner)

  for each l in outer:                                  # (A) ascending by (k,t)
      if key(l) != cand_key:                            # (B) new key group
          cand     <- EMPTY
          cand_key <- key(l)

      while r is not EOF and key(r) <  key(l):          # (C) skip stale keys
          r <- next(inner)

      while r is not EOF and key(r) == key(l)
                         and t(r) <= t(l):              # (D) absorb candidates
          cand <- COPY(r)                               # (E) must copy!
          r    <- next(inner)

      if cand is EMPTY:
          emit l o NULLs                                # left-outer
      else:
          emit l o cand
```

Listing 6.1: ASOF left join, both inputs sorted ascending by (k, t). `cand` is a slot owned by this node.

Five remarks, keyed to the comments:

(A) Both sides are sorted. The planner guarantees this by requiring pathkeys (Chapter 7); the executor may assume it.

(B) Key groups are contiguous. Because the sort is on (k, t) with k leading, all rows of a key arrive together. A key change resets the candidate — forgetting this reset is the classic bug, and it leaks matches from one symbol to the next.

(C) and (D) The inner cursor never rewinds. Line (C) discards inner keys that the outer has passed; line (D) advances while the inner row still satisfies the match condition. Both loops only ever call `next(inner)`, which is why the total inner work is $O(m)$ across the entire join rather than per outer row.

(E) The candidate must be *copied*. `cand <- COPY(r)` is `ExecCopySlot`, not a pointer assignment. The inner child reuses its output slot on the next `ExecProcNode` call, so a retained pointer becomes a dangling reference to a tuple that has already been overwritten — a bug that manifests as sporadically wrong values rather than a crash, and therefore one to prevent by construction.

Strict operators replace `t(r) <= t(l)` with `t(r) < t(l)` in (D). The "earliest following" variants sort descending, or equivalently reverse the comparison — the loop structure is unchanged.

6.2.1 Why no mark/restore

A merge *equi*-join must handle duplicate join keys on the inner side: when the outer advances to another row with the same key, the inner cursor has to be rewound to the start of that key's run. PostgreSQL implements this with `ExecMarkPos`/`ExecRestrPos` on the inner node, which in turn forces a `Materialize` node when the inner plan cannot rewind cheaply — a significant complication in `nodeMergejoin.c`.

ASOF join needs none of it. By property (P1) each outer row has exactly one match, and that match is *the last candidate absorbed* — a single tuple, already held in `cand`. Consecutive outer rows with the same key continue absorbing from where the previous one stopped; the candidate is only ever extended forward, never re-scanned. The one-tuple buffer replaces the entire mark/restore mechanism.

> **Key idea.** ASOF's "exactly one match" property is not just a semantic curiosity — it is what collapses merge join's rewind machinery into a single remembered tuple. A weaker operator would need the full apparatus.

6.3 Correctness

Invariant 6.1 (Candidate). *At the point where the algorithm emits a row for outer tuple ℓ, either `cand` holds a tuple $r^\star$ with*

$$r^\star \in C(\ell) \quad \textit{and} \quad t_R(r^\star) = \max_{r \in C(\ell)} t_R(r),$$

or `cand` is empty and $C(\ell) = \varnothing$.

Proof. By induction over the outer stream.

Base. Before the first outer tuple, `cand` is empty and `cand_key` is NULL, so line (B) resets and the loops at (C),(D) run from the start of the inner stream.

Step. Assume the invariant held for the previous outer tuple ℓ'. Two cases.

CASE 1: $k(\ell) \neq k(\ell')$. Line (B) empties `cand`. Line (C) advances r past all inner keys less than $k(\ell)$; since the inner stream is sorted, no inner row with key $k(\ell)$ has been consumed and discarded (consumption only happens in (C), for strictly smaller keys, or in (D), for the previous key group). Line (D) then absorbs precisely those rows with key $k(\ell)$ and $t_R \leq t_L(\ell)$, in ascending t_R order, assigning each to `cand`; the last assignment is therefore the maximum. If no such row exists, `cand` remains empty and indeed $C(\ell) = \varnothing$.

CASE 2: $k(\ell) = k(\ell')$. Then `cand` still holds the maximal element of $C(\ell')$. Because the outer stream is sorted on (k, t), $t_L(\ell) \geq t_L(\ell')$, hence $C(\ell') \subseteq C(\ell)$: the candidate set only grows. The rows of $C(\ell) \setminus C(\ell')$ are exactly those with key $k(\ell)$ and $t_L(\ell') < t_R \leq t_L(\ell)$, which are the next rows in the inner stream and are precisely what loop (D) now absorbs, again in ascending order. After the loop `cand` holds the maximum over $C(\ell') \cup (C(\ell) \setminus C(\ell')) = C(\ell)$, since the previous candidate was maximal over $C(\ell')$ and each absorbed row is compared by recency. If both are empty, $C(\ell) = \varnothing$. □

Theorem 6.2 (Algorithm correctness)**.** *Given inputs sorted ascending on (k, t), the algorithm of §6.2 computes the* ASOF *left join of Definition 1.1.*

Proof. The outer loop visits each $\ell \in L$ exactly once and emits exactly one row for it, so the output has $|L|$ rows as required by (P1). By Invariant 6.1, the emitted row is $\ell \circ m(\ell)$ when $C(\ell) \neq \varnothing$ and ℓ NULL-extended otherwise, which is Definition 1.1. The inner variant differs only in suppressing the NULL-extended rows. □

Remark 6.3 (The monotonicity requirement)**.** Case 2 of the induction uses $t_L(\ell) \geq t_L(\ell')$ — the outer stream must be sorted *within* a key. If it were not, C could shrink between consecutive outer rows and the retained candidate would be too recent. This is precisely why the planner must enforce a pathkey on the outer side too, not only on the inner (§7.1); sorting only the inner side gives a plausible-looking plan that is quietly wrong.

6.4 Complexity

Let $n = |L|$, $m = |R|$.

$$T_{\text{asof}} = \underbrace{O(n \log n) + O(m \log m)}_{\text{sorts, if needed}} + \underbrace{O(n + m)}_{\text{merge pass}}$$

and $O(n + m)$ alone when indexes already supply both orders. Compare with $O(n \log m)$ random descents for the LATERAL plan. The asymptotic gap is modest; the *constant* is where the win lives:

- the merge pass is sequential, so it is prefetch- and cache-friendly, whereas index descents are random;
- it performs one executor node invocation per tuple, not one subplan execution per outer row;
- sorts are performed once, in bulk, with `work_mem`-sized runs, and are skipped entirely when an index on (k, t) exists.

Memory is $O(1)$ beyond the sorts: exactly one candidate tuple, regardless of data size — in contrast to a hash-based strategy, which is not even applicable here because the predicate is an inequality.

6.5 A worked example

Figure 6.1 traces the algorithm on one key group.

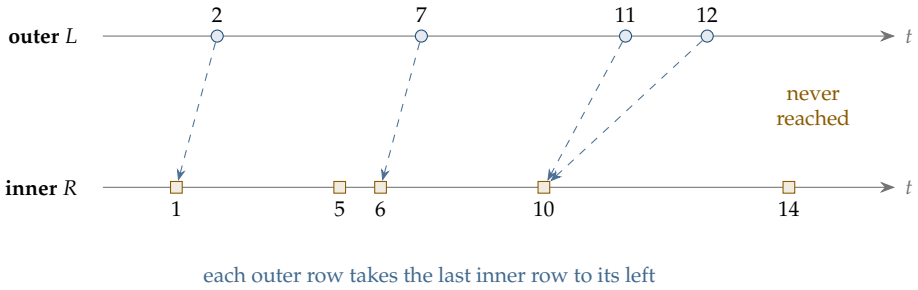


Figure 6.1: One key group. Outer timestamps $2, 7, 11, 12$; inner $1, 5, 6, 10, 14$. The inner cursor moves left to right exactly once. Note that $t = 5$ is absorbed and then superseded by $t = 6$ — it is a candidate briefly — and that outer rows 11 and 12 share the candidate 10 without any rewind.

Step	Outer t_L	Inner absorbed	`cand` after
1	2	1	1
2	7	5, then 6	6
3	11	10	10
4	12	— (next is 14, fails $t_R \leq 12$)	10

Table 6.1: Trace of Figure 6.1. Total inner advances: 5. A LATERAL plan would perform 4 independent index descents instead.

6.6 Exercises

1. Run the algorithm by hand on Figure 6.1 with the strict operator $<$. Which output row changes?

2. Modify the pseudocode for `ASOF JOIN` (inner). Where exactly does the change go, and does Invariant 6.1 still hold?

3. Suppose the outer side were *not* sorted within a key. Give a three-row example producing a wrong answer, illustrating the monotonicity remark.

4. What is the peak memory of the merge pass, excluding sorts? Compare with `nodeMergejoin.c`'s requirement when the inner side needs materialising.

5. Why can the inner cursor safely skip past a key that the outer never visits? Which line does that, and what would happen if it compared with <= instead of <?

Chapter 7

Making the Planner Choose It

A correct executor node is useless if no `Path` leads to it. This chapter covers the three things the planner must learn: the required sort order, the legality constraints, and the cost.

7.1 Pathkeys: the planner's model of order

The merge algorithm's precondition is that both inputs arrive sorted ascending on $(k_1, \ldots, k_j, t)$ — *equality keys first, match column last*. In planner terms this is a list of `PathKeys`, and the mechanism for obtaining it is the same one merge join uses:

1. Build the required `PathKey` list for each side with the `make_pathkeys_for_sortclauses` family, using the `btree` opfamily established during Phase 1 validation.
2. For each candidate child `Path`, compare its `pathkeys` against the requirement with `pathkeys_contained_in()`. A path whose existing order has the requirement as a *prefix* needs no sort.
3. Otherwise wrap the child in `create_sort_path()`, paying the sort cost.

> **Key idea.** This step is where the performance claim is cashed. An index on `quotes (symbol, ts)` produces an `Index Scan` path whose pathkeys already match, so *both* sorts disappear and the join degenerates to a pure $O(n + m)$ streaming pass over two index scans. Without the pathkey machinery we would sort unconditionally and give back much of the advantage.

Note the ordering requirement is on (k, t) with k *leading*: the algorithm needs key groups to be contiguous (remark (B) of §6.2). An index on `(ts, symbol)` does not help, which is a useful thing to be able to explain to a user staring at an unexpected `Sort` node.

7.2 Path generation

`add_paths_to_joinrel()` in `optimizer/path/joinpath.c` is where each join strategy proposes its paths. For `JOIN_ASOF` the ASOF path is the *only* candidate:

- **no hash join** — hashing implements equality, and the match condition is an inequality;
- **no nested loop** — it would have to re-search per outer row, which is the very thing we are replacing (and the Phase 1 rewrite already covers that plan shape);
- **merge-style only**, with sorts added as needed.

The path itself is created by a new `create_asofjoin_path()` in `optimizer/util/pathnode.c`, modelled closely on `create_mergejoin_path()`.

7.3 Why ASOF constrains join order

Property (P2) — non-commutativity — has teeth in the optimiser. PostgreSQL's planner explores join orders freely for inner joins, because $A \bowtie B \equiv B \bowtie A$ and $(A \bowtie B) \bowtie C \equiv A \bowtie (B \bowtie C)$. Outer joins already break those identities, and `optimizer/path/joinrels.c` contains the machinery that restricts them: `join_is_legal()` decides whether a given pair may be joined at all, and `have_join_order_restriction()` tells the planner not to give up on a join order it would otherwise prune.

ASOF must join the same restrictions:

- **Not commutable.** L must remain the outer side. The identity of "which side is probed" is part of the operator's meaning, not an implementation detail.
- **Not freely reassociable.** Pushing another join inside an ASOF join can change which rows exist on the inner side and therefore which row is nearest — the same hazard that makes outer-join reassociation conditional.
- **Qual placement.** A filter on the inner relation changes the candidate set, hence the answer. It must not be pushed below the ASOF join unless it was written there, exactly as for the nullable side of an outer join.

The practical approach is to find every `switch (jointype)` in the optimiser and handle `JOIN_ASOF` deliberately. Building with `-Wswitch` turns this from an exercise in grep into a compiler-enforced checklist — a worthwhile technique in general when adding a variant to a widely-switched enum.

7.4 The cost model

PostgreSQL costs a path in two parts: `initial_cost_xxx` (cheap, used to prune) and `final_cost_xxx` (exact, for survivors). For ASOF:

$$\text{startup} = C_{\text{sort}}(L) + C_{\text{sort}}(R) \quad \text{(either term zero if presorted)} \tag{7.1}$$

$$\text{total} = \text{startup} + (n+m)\, c_{\text{tuple}} + n\, c_{\text{match}} + n\, c_{\text{proj}} \tag{7.2}$$

where c_{match} is the cost of evaluating the match comparison and the key equality, and c_{proj} the projection cost. Three observations:

1. **The sorts dominate the startup cost**, which means an ASOF join is a poor choice under a `LIMIT` unless the inputs are presorted — the planner needs the startup/total split to make that trade-off correctly. This is exactly the behaviour of merge join, and the reason PostgreSQL models startup cost at all.

2. **There is no rescan term.** Merge join's cost function includes an allowance for re-reading inner tuples after a restore; ASOF has none (§6.2), so the linear term is clean. Getting this right matters: over-costing the join makes the planner prefer a nested loop and the whole exercise is wasted.

3. **Output cardinality is known exactly** for the left variant: n rows, by (P1). This is a rare luxury — most join estimates are the product of a selectivity guess — and it makes everything above an ASOF join easier to plan. For the inner variant, estimate $n \cdot p$ where p is the fraction of left rows having a predecessor, which can be approximated from the two relations' histograms on t (specifically, the fraction of L's distribution lying above $\min t_R$ per key).

7.5 The GUC

`enable_asofjoin` (default `on`) is added to `utils/misc/guc_tables.c` and `postgresql.conf.sample`. Switching it `off` reverts to the Phase 1 rewrite. This is not merely a safety valve: it is the instrument that makes the Chapter 9 benchmark and differential test possible at all, since it lets the same query text run through both implementations in the same session.

7.6 Exercises

1. Explain why `(ts, symbol)` is the wrong index for an ASOF join on `symbol` and why `(symbol, ts)` is right.
2. The required pathkeys put equality keys first. Could the algorithm be reformulated to allow t first? What would it cost?
3. Under what circumstances would a nested-loop ASOF plan actually be cheaper than the merge plan? Should we offer it?
4. Why does the planner need both `initial_cost` and `final_cost` functions rather than computing the exact cost once?

Chapter 8

Correctness Corners

Most of the defect risk in a feature like this sits in a handful of corners. Each gets a rule and a regression test.

NULL in the match column. A NULL t on either side makes the comparison UNKNOWN, so the row is not a candidate. In Phase 1 this falls out of the subquery's `WHERE`. In Phase 2 it must be explicit: the merge comparison has to treat NULL as non-matching rather than as an ordering extreme. Note the trap — in a *sort*, NULLs are ordered (nulls last by default for ascending), so they will arrive at the end of the stream and must be skipped rather than absorbed as candidates.

NULL in a partition key. Treated as never-equal, consistent with =. Both phases get this right for free if the key comparison uses the ordinary equality operator; the trap is using a *sort* comparison instead, under which two NULLs compare equal. This is the one place where "sorted together" and "equal" genuinely diverge, and it is worth a dedicated test.

Ties on t_R. Implementation-defined, documented, and tested only to the extent that both phases agree. See the remark in Chapter 2.

Empty inputs. Empty R: all left rows NULL-extended (left) or no rows (inner). Empty L: no rows, and — worth asserting — the inner side should ideally not be read at all. Both are one-line tests and both have caught real bugs in similar operators.

Strict versus non-strict operators. The only difference is whether equal timestamps match. In Phase 2 this is a single comparison operator in loop (D); getting it backwards produces an off-by-one-row error that is invisible unless the test data contains exactly-equal timestamps. The test data must therefore contain them deliberately.

Rescan. `ExecReScanAsofJoin` must reset the candidate slot, the key, and both children. Without the candidate reset, an ASOF join on the inner side of a nested loop returns the *previous* outer iteration's match on its first row — a subtle, data-dependent wrong answer. Test it under a correlated subquery.

Memory. The candidate slot must live in a context that is reset per key group, or at least not leaked per tuple. A 10^7-row query is the right smoke test: memory should be flat.

Parallelism. The node is marked parallel-unsafe initially. A single ordered pass over a shared inner stream does not partition trivially; the natural parallelisation is per-key-range, which requires a partitioned sort and is out of scope. Mark it, assert it, and note the follow-up.

8.1 Exercises

1. Construct test data in which the strict/non-strict distinction changes the result for exactly one row.
2. Why does the NULL-in-key rule require an equality operator rather than the sort comparator? Write the two-row example that distinguishes them.
3. Design the correlated-subquery test that would catch a missing candidate reset in `ExecReScanAsofJoin`.

Chapter 9

Measuring It

9.1 Reading the plans

The three plan shapes to recognise:

```
Nested Loop Left Join
  ->  Seq Scan on trades t
  ->  Limit
        ->  Index Scan Backward using quotes_symbol_ts_idx on quotes q
              Index Cond: ((symbol = t.symbol) AND (ts <= t.ts))
```

Listing 9.1: LATERAL workaround — per-row index descent.

```
Asof Left Join
  Match Condition: (t.ts >= q.ts)
  Asof Keys: (t.symbol = q.symbol)
  ->  Sort
        Sort Key: t.symbol, t.ts
        ->  Seq Scan on trades t
  ->  Sort
        Sort Key: q.symbol, q.ts
        ->  Seq Scan on quotes q
```

Listing 9.2: Native, inputs not presorted — two sorts, one pass.

```
Asof Left Join
  Match Condition: (t.ts >= q.ts)
  Asof Keys: (t.symbol = q.symbol)
  ->  Index Scan using trades_symbol_ts_idx on trades t
  ->  Index Scan using quotes_symbol_ts_idx on quotes q
```

Listing 9.3: Native, indexes supply the order — the goal.

The presence or absence of the `Sort` nodes is the single most informative thing in these plans, and it is why `EXPLAIN` output is a deliverable in its own right.

9.2 Benchmark design

What to vary. Row counts 10^5 / 10^6 / 10^7; number of distinct keys (1, 50, 10^4 — key cardinality changes how much of the inner stream each outer row skips); index present or absent; left versus inner; match density (fraction of left rows with a predecessor).

What to compare. Three implementations on identical data: hand-written LATERAL, Phase 1 rewrite (expected to be *identical* — any gap is a rewrite bug, not a finding), and Phase 2 native with `enable_asofjoin` on.

Pitfalls.

- **Cold cache.** First run reads from disk; subsequent runs do not. Warm up, or report both, but never mix them in one table.
- **Planner noise.** Run `ANALYZE` before timing, and check with `EXPLAIN` that the plan is the one you think you are measuring. A benchmark that silently measured a different plan is the most common way to get a wrong answer here.
- `EXPLAIN ANALYZE` **overhead.** Per-tuple instrumentation can add tens of percent, and it falls more heavily on plans with more node invocations — i.e. unfairly on the LATERAL plan. Time with `\timing` for the headline numbers; use `EXPLAIN ANALYZE` for diagnosis.
- `work_mem`**.** The native plan's sorts spill to disk below a threshold and the measurement falls off a cliff. Report the setting.
- **Index-only scans.** A covering index can make the LATERAL plan much faster than expected. Note whether the index covers the projected columns.

9.3 Differential testing

The strongest correctness tool the project has is that Phase 1 and Phase 2 compute the same function by entirely different means. The harness runs the whole query corpus twice — `enable_asofjoin = off`, then `on` — and diffs. Any difference is a bug in one of them.

```
-- equivalence check, per query
SET enable_asofjoin = off;  CREATE TEMP TABLE a AS <query>;
SET enable_asofjoin = on;   CREATE TEMP TABLE b AS <query>;
SELECT count(*) FROM ((TABLE a EXCEPT ALL TABLE b)
                      UNION ALL (TABLE b EXCEPT ALL TABLE a)) d;  -- must be 0
```

Extend this with randomised inputs: generated tables with random key cardinality, random NULL density, random tie frequency, and a randomly chosen operator. Property-based testing finds the corners of Chapter 7 far more reliably than hand-written cases, because it does not share the implementer's blind spots.

9.4 Exercises

1. Why is `EXCEPT ALL` in both directions necessary, rather than a single `EXCEPT`?

2. The equivalence check above compares multisets. How should it be amended for queries whose results differ legitimately due to ties?

3. Predict the shape of the native-versus-LATERAL speedup curve as key cardinality rises from 1 to 10^4, and justify your prediction in terms of §6.2.

Appendix A

Source File Map

Every PostgreSQL file the project touches, keyed to the work plan's issue IDs (`plan.md`).

File	Role	Issues
Parser		
`include/parser/kwlist.h`	keyword registration	P1-A1
`backend/parser/gram.y`	grammar productions	P1-A1/A2
`include/nodes/primnodes.h`	`JoinExpr` fields	P1-A3
`backend/parser/parse_clause.c`	`transformFromClauseItem`	P1-B1
`backend/parser/parse_asof.c` *(new)*	the LATERAL rewrite	P1-B2
Nodes & deparse		
`backend/nodes/gen_node_support.pl`	copy/equal/out/read generation	P1-A3, P2-C2
`backend/utils/adt/ruleutils.c`	`pg_get_viewdef` deparse	P1-A3, P2-C2
Planner		
`include/nodes/nodes.h`	`JOIN_ASOF` in `JoinType`	P2-B1
`backend/optimizer/path/joinrels.c`	join legality, order restrictions	P2-B1
`backend/optimizer/path/joinpath.c`	path generation	P2-B2
`backend/optimizer/util/pathnode.c`	`create_asofjoin_path`	P2-B2
`backend/optimizer/path/costsize.c`	cost functions	P2-B3
`backend/utils/misc/guc_tables.c`	`enable_asofjoin`	P2-B3
`backend/optimizer/plan/createplan.c`	`create_asofjoin_plan`	P2-A1
`include/nodes/plannodes.h`	`AsofJoin` plan node	P2-A1
Executor		
`backend/executor/nodeAsofjoin.c` *(new)*	the merge algorithm	P2-A2
`include/nodes/execnodes.h`	`AsofJoinState`	P2-A2
`backend/executor/execProcnode.c`	node dispatch	P2-A1
`backend/executor/execAmi.c`	`ExecReScan`	P2-A3
Surface & tests		
`backend/commands/explain.c`	plan output	P2-C1
`test/regress/sql/asof_join.sql`	regression suite	P1-C2
`doc/src/sgml/ref/select.sgml`	user documentation	P1-C3

Appendix B

Glossary

`Path`
: One candidate way of producing a relation, with cost and sort order. The planner enumerates paths and keeps the cheapest.

`PathKey`
: Canonical representation of a sort order, used to decide whether a sort can be skipped.

`RelOptInfo`
: Planner's record of a relation — base or join — and its paths.

`TupleTableSlot`
: Holder for one tuple in the executor. Reused by the producing node, hence the copy requirement in §6.2.

`LATERAL`
: `FROM`-clause modifier permitting a subquery to reference earlier `FROM` items; evaluated per outer row.

`Desugaring`
: Defining a new construct by translation into existing ones. Phase 1's technique.

`Volcano model`
: Iterator-based execution: every node exposes `next()`, pulling tuples from children on demand.

`Mark/restore`
: Merge join's mechanism for rewinding the inner side over duplicate keys. Not needed by ASOF — see §6.2.

`opfamily`
: Catalogued group of related operators (e.g. `btree` comparison operators for a type), which is how PostgreSQL knows a type has a usable total order.

`GUC` "Grand Unified Configuration" — a PostgreSQL run-time setting, e.g. `enable_asofjoin`.